

conv3d#

<https://pytorch.org/docs/stable/generated/torch.nn.quantized.functional.conv3d.html>

Description:

Applies a 3D convolution over a quantized 3D input composed of several input planes. See Conv3d for details and output shape. input – quantized input tensor of shape $(\text{minibatch}, \text{in_channels}, \text{iD}, \text{iH}, \text{iW})$ (minibatch , in_channels , iD , iH , iW) weight – quantized filters of shape $(\text{out_channels}, \text{in_channelsgroups}, \text{kD}, \text{kH}, \text{kW})$ (out_channels , $\frac{\text{in_channels}}{\text{groups}}$, kD , kH , kW) ($\text{out_channels}, \text{groupsin_channels}, \text{kD}, \text{kH}, \text{kW}$) bias – non-quantized bias tensor of shape (out_channels) (out_channels) (out_channels). The tensor type must be torch.float.

Function Signature

```
class torch.nn.quantized.functional.conv3d(input, weight,
bias, stride=1, padding=0, dilation=1, groups=1,
padding_mode='zeros', scale=1.0, zero_point=0,
dtype=torch.quint8)[source]#
```

Parameters

Code Examples

```
>>> from torch.nn.quantized import functional as qF
>>> filters = torch.randn(8, 4, 3, 3, 3, dtype=torch.float)
>>> inputs = torch.randn(1, 4, 5, 5, 5, dtype=torch.float)
>>> bias = torch.randn(8, dtype=torch.float)
>>>
>>> scale, zero_point = 1.0, 0
>>> dtype_inputs = torch.qint8
>>> dtype_filters = torch.qint8
>>>
>>> q_filters = torch.quantize_per_tensor(filters, scale,
    zero_point, dtype_filters)
>>> q_inputs = torch.quantize_per_tensor(inputs, scale,
    zero_point, dtype_inputs)
>>> qF.conv3d(q_inputs, q_filters, bias, padding=1, scale=scale,
    zero_point=zero_point)
```

Detailed Documentation

Documentation

Applies a 3D convolution over a quantized 3D input composed of several input planes.

See Conv3d for details and output shape.

input – quantized input tensor of shape (minibatch,in_channels,iD,iH,iW)
(\text{minibatch} , \text{in_channels} , iD , iH , iW)(minibatch,in_channels,iD,iH,iW)

weight – quantized filters of shape (out_channels,in_channelsgroups,kD,kH,kW)

$(\text{out_channels}, \frac{\text{in_channels}}{\text{groups}}, \text{kD}, \text{kH}, \text{kW})$
(out_channels, groups, in_channels, kD, kH, kW)

bias – non-quantized bias tensor of shape (out_channels)(\text{out_channels})(out_channels). The tensor type must be torch.float.

stride – the stride of the convolving kernel. Can be a single number or a tuple (sD, sH, sW). Default: 1

padding – implicit paddings on both sides of the input. Can be a single number or a tuple (padD, padH, padW). Default: 0

dilation – the spacing between kernel elements. Can be a single number or a tuple (dD, dH, dW). Default: 1

groups – split input into groups, in_channels\text{in_channels}in_channels should be divisible by the number of groups. Default: 1

padding_mode – the padding mode to use. Only “zeros” is supported for quantized convolution at the moment. Default: “zeros”

scale – quantization scale for the output. Default: 1.0

zero_point – quantization zero_point for the output. Default: 0

dtype – quantization data type to use. Default: torch.quint8